

Technical Architecture &

Network Integrations Manual (Google Cloud Edition)

Document:	ATLAS Travel Club LLC — Engineering Standards Reference
Standard:	100% Google Cloud Ecosystem (Cloud Run, Vertex AI, Cloud SQL)
Audience:	Engineering, DevOps, and Security Reviewers
Date:	September 19, 2026
Classification:	Confidential — Subject to Mutual NDA

Table of Contents

1. Architecture Overview

3

1.1 Server & Deployment

3

1.2 Database & State

3

1.3 Artificial Intelligence (Vertex AI & Gemini)

4

1.4 Secrets & Environment Management

4

1.5 Network Integrations Summary

5

1.6 Security & Compliance Architecture

5

1. Architecture Overview

The ATLAS Travel Club LLC platform is standardized 100% on the Google Cloud Ecosystem. This manual documents the canonical deployment, database, AI, and secrets architecture used across all environments (development, staging, production). All engineering decisions must conform to this standard; deviations require written approval from the Manager and a documented technical justification.

1.1 Server & Deployment

All application workloads are deployed to Google Cloud Run as stateless containerized services. The canonical production deployment command is:

```
gcloud run deploy atlas-travel-club \
  --image gcr.io/atlas-prod/app \
  --region us-central1 \
  --platform managed \
  --memory 2Gi --cpu 2 \
  --min-instances 1 --max-instances 50 \
  --concurrency 80 \
  --service-account atlas-svc@atlas-prod.iam.gserviceaccount.com \
  --set-secrets=HOTELBEDS_API_KEY=hotelbeds-key:latest \
  --set-secrets=HOTELBEDS_SECRET=hotelbeds-secret:latest \
  --set-secrets=RATEHAWK_KEY_ID=ratehawk-key-id:latest \
  --set-secrets=RATEHAWK_API_KEY=ratehawk-api-key:latest \
  --set-secrets=DUFFEL_ACCESS_TOKEN=duffel-token:latest \
  --set-secrets=STRIPE_SECRET_KEY=stripe-secret:latest \
  --vpc-egress private-ranges-only \
  --add-cloudsql-instances=atlas-prod:us-central1:atlas-pg16
```

Continuous integration is configured via Google Cloud Build connected to the Company's GitHub repository. The build pipeline executes the following stages on every push to main: (1) install dependencies via Bun; (2) run ESLint and TypeScript type-check; (3) execute unit tests; (4) build the production container image; (5) push the immutable image to `gcr.io/atlas-prod/app:$SHORT_SHA`; (6) deploy to Cloud Run staging; (7) on manual approval, promote to production with traffic splitting for canary validation.

1.2 Database & State

The primary database is Google Cloud SQL for PostgreSQL (v16), instance `atlas-prod:us-central1:atlas-pg16`, configured with high availability (regional persistent disks), automated backups (7-day retention with point-in-time recovery), and private IP only (no public IP exposed). All application connections route through the Cloud SQL Auth Proxy, which provides secure zero-IP-exposure database tunneling using short-lived IAM-derived OAuth tokens rather than static database passwords.

```
# Cloud SQL Auth Proxy – local development pattern
./cloud-sql-proxy \
  atlas-prod:us-central1:atlas-pg16 \
  --credentials-file=$GOOGLE_APPLICATION_CREDENTIALS \
  --auto-iam-authn

# Connection string (no IP, no password – IAM-derived token only)
DATABASE_URL=postgresql://atlas-svc@atlas?host=/cloudsql/atlas-prod:us-central1:atlas-pg16
```

Reservation archives, member-uploaded documents, and audit logs are stored in Google Cloud Storage (GCS) buckets with bucket-level retention policies, customer-managed encryption keys (CMEK) via Cloud KMS, and uniform bucket-level access. Public access is permanently prevented at the bucket policy level.

1.3 Artificial Intelligence (Vertex AI & Gemini)

All large language model inference is routed through Google Cloud Vertex AI. No external LLM endpoints (OpenAI, Anthropic, etc.) are permitted in production. The production configuration uses Gemini 1.5 Pro for deep reasoning tasks (itinerary planning, complex booking orchestration, multi-supplier rate analysis) and Gemini 2.0 Flash for low-latency conversational turns (instant chat replies, quick queries, FAQ responses).

```
// Vertex AI integration via @google-cloud/vertexai SDK
import { VertexAI } from '@google-cloud/vertexai';

const vertexAI = new VertexAI({
  project: 'atlas-prod',
  location: 'us-central1',
});

// Deep reasoning – itinerary planning, complex orchestration
const proModel = vertexAI.getGenerativeModel({
  model: 'gemini-1.5-pro-002',
  generationConfig: { temperature: 0.4, maxOutputTokens: 4096 },
  tools: [{ functionDeclarations: BEDBANK_TOOLS }], // RateHawk, Hotelbeds, Duffel
});

// Low-latency conversational turns – instant chat replies
const flashModel = vertexAI.getGenerativeModel({
  model: 'gemini-2.0-flash-exp',
  generationConfig: { temperature: 0.7, maxOutputTokens: 1024 },
});
```

The ATLAS AI Concierge reasoning pipeline utilizes Vertex AI Function Calling to query RateHawk, Hotelbeds, and Duffel APIs in real-time. When a member sends a natural-language request (e.g., “book a 5-night stay in Tokyo under \$2,000 total”), the model emits a structured function call that the ATLAS backend executes against the appropriate bedbank. The result is returned to the model, which synthesizes a final response with booking links. This pattern keeps all live inventory queries server-side and gated behind authenticated member sessions, preserving the closed-loop rate parity compliance posture.

1.4 Secrets & Environment Management

All credentials — including HOTELBEDS_API_KEY, HOTELBEDS_SECRET, RATEHAWK_KEY_ID, RATEHAWK_API_KEY, DUFFEL_ACCESS_TOKEN, STRIPE_SECRET_KEY, STRIPE_ISSUING_WEBHOOK_SECRET, APPLE_PASS_CERT_PEM, APPLE_PASS_KEY_PEM, AIRALO_CLIENT_SECRET, SHERPA_API_KEY, PRUVO_PARTNER_SECRET, TELEGRAM_BOT_TOKEN, and WHATSAPP_TOKEN — are stored as versioned secrets in Google Secret Manager. Secrets are never written to source control, never logged, and never baked into container images. At runtime, Cloud Run mounts each secret directly into the container environment as an environment variable via the `--set-secrets` deployment flag.

```
# Create a new secret version in Google Secret Manager
gcloud secrets create hotelbeds-api-key \
  --replication-policy=automatic \
  --data-file=- <<< "$HOTELBEDS_API_KEY"

# Grant Cloud Run service account access (least-privilege IAM)
gcloud secrets add-iam-policy-binding hotelbeds-api-key \
  --member=serviceAccount:atlas-svc@atlas-prod.iam.gserviceaccount.com \
  --role=roles/secretmanager.secretAccessor

# Reference in Cloud Run deployment (see deploy command in Section 1.1)
```

1.5 Network Integrations Summary

Supplier / Service	Endpoint	Secret Reference
Hotelbeds (APItude)	api.hotelbeds.com	hotelbeds-key, hotelbeds-secret
RateHawk (ETG)	api.worldota.net/api/b2b/v3	ratehawk-key-id, ratehawk-api-key
Duffel (NDC Flights)	api.duffel.com	duffel-token
Stripe Issuing	api.stripe.com	stripe-secret, stripe-issuing-webhook
Airalo (eSIM)	partner-api.airalo.com/v2	airalo-client-id, airalo-client-secret
Sherpa (Visa Intel)	requirements-api.joinsherpa.com/v2	sherpa-api-key
Pruvo (Price Drop)	partner.pruvo.com	pruvo-partner-secret
WhatsApp Cloud API	graph.facebook.com/v18.0	whatsapp-token, whatsapp-verify
Telegram Bot API	api.telegram.org	telegram-bot-token

1.6 Security & Compliance Architecture

Google Cloud Armor is deployed at the edge to provide enterprise-grade DDoS mitigation and bot protection, forming the foundation of the proprietary Rate Parity Compliance Shield. Cloud Armor security policies enforce rate limiting on unauthenticated endpoints, block known scraper IP ranges, and apply X-Robots-Tag: noindex at the CDN layer. All ingress traffic terminates at Cloud Load Balancing with managed TLS certificates; no traffic reaches Cloud Run containers directly. Cloud Audit Logs captures all IAM and data-access events for 365 days, exported to a dedicated GCS bucket for compliance retention.